Tecnológico de Monterrey

# 14. DTMs y Modelos de clasificación

## Ciencia de datos para la toma de decisiones II


Jorge
Juvenal
Campos Ferreira

juvenal.campos@tec.mx

# *Matríz documento-término*

Una ⭐ **matriz documento -término (*Document-term Matrix*, o DTM)** es una matríz en la que cada columna representa un término, palabra o token, cada fila representa un documento (artículo, tweet o párrafo) y cada celda contiene el número de veces que ese término aparece en ese documento (frecuencia) o una ponderación (como TF-IDF).

**Tokens**

| Doc\Término | economía | inflación | crecimiento |
|---|---|---|---|
| **Doc1** | 3 | 1 | 0 |
| **Doc2** | 2 | 0 | 4 |
| **Doc3** | 1 | 2 | 1 |

# Función *tidytext::cast_dtm()*

Es la función que nos ayuda a pasar de una tabla tidy de palabras-documento (la que generamos con el *unnest_token*() o con la lematización) a una matríz término-documento.

```
matriz_dtm <- texto_token %>%
  cast_dtm(document = id,
           term = palabras,
           value = n,
           weighting = tm::weightTfIdf)
```

Como se ve

```
<<DocumentTermMatrix (documents: 690, terms: 11)>>
Non-/sparse entries: 759/6831
Sparsity           : 90%
Maximal term length: 11
Weighting          : term frequency - inverse document frequency (normalized) (tf-idf)
```

Convertida en *matríz* y luego en *tibble*

```
> matriz_dtm %>% as.matrix() %>% as_tibble()
# A tibble: 690 × 11
   gente violencia videojuegos juegos impuestos hacer
   <dbl>     <dbl>       <dbl>  <dbl>     <dbl> <dbl>
 1 0         0           0      0           0   0
 2 0.554     0           0      0           0   0
 3 0         0.541       0      0           0   0
 4 0         0.0971      0.106  0.0493      0   0
 5 0         0           0      0           3.13 0
 6 0         0           0      0           0   0.269
 7 0         0           0      0           0   0
 8 0.117     0           0.167  0           0   0.122
 9 0         0           0      0           0   0
10 0         0           0      0           0   0
```

# Función *tidytext::cast_dtm()*

**Argumentos**:

- **document_id**: La columna que identifica al documento.
- **term**: la columna que contiene las palabras
- **value**: la columna que contiene los valores de las celdas de la matríz. Pueden ser frecuencias o ponderaciones *tf-idf*.
- **weighting:** Ponderación de la matríz (puede ser frecuencia o tf-idf)

```
matriz_dtm <- texto_token %>%
   cast_dtm(document = id,
            term = palabras,
            value = n,
            weighting = tm::weightTfIdf)
```

# Matríces dispersas

Es una matríz en la que **la mayoría de sus elementos son cero.**

- Si una matríz es 99% dispersa, significa que el 99% de sus elementos es cero.
- Esta característica de **dispersión** es una característica muy común de las matríces documento término, ya que hay tantas palabras en los vocabularios de los corpus que es muy común que haya muchos ceros en los vectores que representan a los documentos.

| gente | violencia | videojuegos | juegos | impuestos | hacer | ms | dinero | impuesto | gobierno | pas |
|---|---|---|---|---|---|---|---|---|---|---|
| <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0.554 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0.541 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0.0971 | 0.106 | 0.0493 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 3.13 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0.269 | 0.182 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0.117 | 0 | 0.167 | 0 | 0 | 0.122 | 0.0827 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

# Matríces dispersas p.2

En procesamiento de texto, las matrices dispersas **son la norma:**

- Un **vocabulario** puede tener **decenas de miles de palabras**.
- Pero cada documento solo usa unas **cuantas centenas**.
- Si guardas todos los ceros, **desperdicias mucha memoria** y **tiempo de cómputo**. Tenemos que hacer algo con ellas.

# Matríces dispersas p.2

**La métrica de la dispersión**, que (en este caso) es una métrica asociada con las matrices de documento término, **se puede reducir** con diversas funciones, como la función ***tm::removeSparseTerms()***.

Con esta función, **podemos reducir la dispersión** en un tanto por ciento, descartando (de forma arbitraria) información dentro de nuestra matríz de análisis.

Decidir sobre la dispersión de la matriz depende de cuántos términos están en tu matriz y qué tan rápida es tu computadora.

# Ejercicio 01 - Creando la dtm

## Ejercicio

1. Cargue los datos del archivo **01_datos/ comentarios_etiquetados.xlsx.** Explore la base resultante.
2. Haga el preprocesamiento de los datos.
3. Tokenice por palabras y elimine palabras comunes (*stopwords*)
4. Obtenga la matríz documento-término de los comentarios.

# ★*Modelos de clasificación*

El modelado de clasificación es el proceso de **aprendizaje supervisado** de entrenar algoritmos para **predecir categorías o clases** de nuevos documentos **basándose en patrones aprendidos** de documentos **etiquetados**.

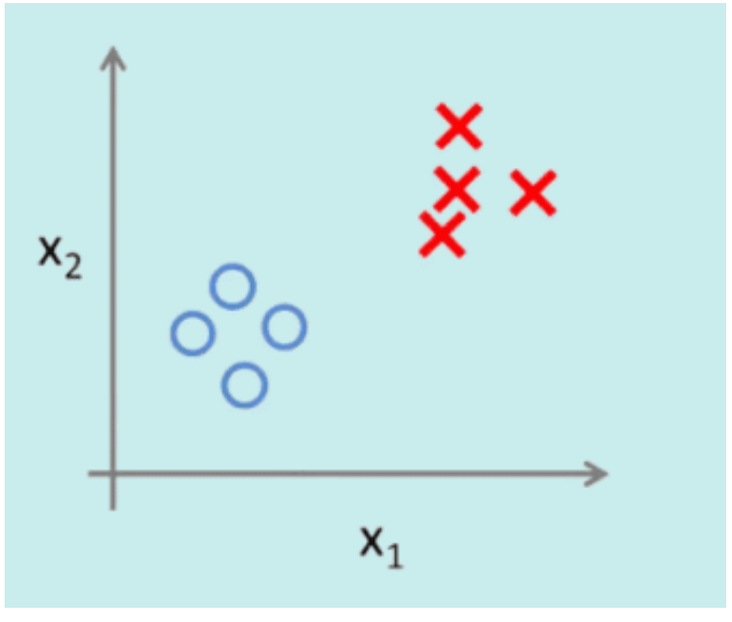
Es una de las aplicaciones más prácticas del análisis de texto en el mundo real.

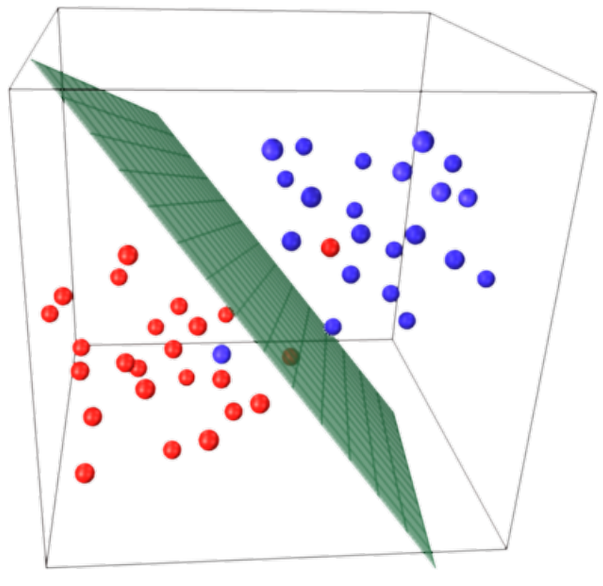
**Características del modelado de clasificación:**

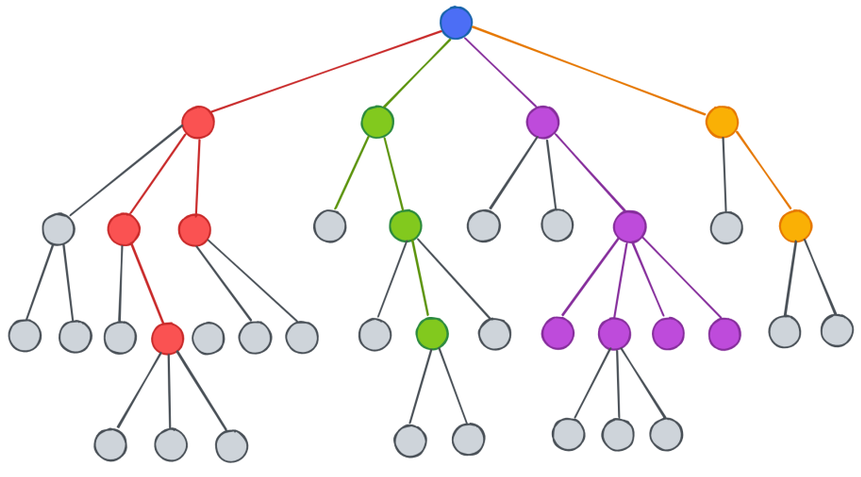
- Enfoque de aprendizaje supervisado
- Distingue entre diferentes categorías
- Requiere datos etiquetados para entrenamiento
- Puede predecir categorías de nuevos documentos

# *Algunos modelos de clasificación:*

- Regresión logística
- Arboles de decisión, bosques aleatorios,
- Xgboost
- Redes Neuronales
- Support Vector Machines
- Entre otros.





# Pasos básicos para usar texto en modelos de Machine Learning

1. Limpia/prepara los datos
2. Crea una base de entrenamiento y una base de prueba/ validación
3. Entrena el modelo en la base de entrenamiento
4. Analiza la exactitud del modelo en la base de prueba/ validación
5. Montamos el modelo en una aplicación y lo utilizamos (producción).

# O como vimos el semestre pasado:

1. Primero cargamos los datos ***etiquetados*** y exploramos los datos (verificamos que la etiqueta sea una variable categórica o factor, que los datos estén balanceados, etc).
2. Partimos los datos en base de entrenamiento y base de prueba.
3. Generamos la receta de feature-engineering y preprocesamiento (con la librería *textrecipes*)
4. Aplicamos la receta para preparar 😋 la dtm de entrenamiento
5. Declaramos el modelo a usar, junto con sus parámetros.
6. Ajustamos o entrenamos el modelo con los datos de entrenamiento.
7. Predecimos en la base de entrenamiento las etiquetas.
8. Obtenemos las métricas de evaluación correspondientes
9. Hacemos la evaluación con la base de prueba.
10. Si es un modelo considerablemente bueno, pasamos a su implementación en producción. Si no, iteramos o hacemos cambios para mejorarlo.

# Ejemplos de *textrecipes*

```
# Receta de preprocesamiento (tidymodels + textrecipes) para convertir texto
# crudo en una matriz TF-IDF que pueda alimentar a un modelo de clasificación

receta <- recipe(etiqueta ~ texto, data = entrenamiento) |>
  # Declara el modelo: `etiqueta` es la variable a predecir, `texto` el predictor.
  # `entrenamiento` solo sirve para que la receta conozca tipos y nombres de columnas;
  # ningún paso se ejecuta todavía (se ejecutan al hacer prep() / bake() o dentro de un workflow).

  step_mutate(texto = str_remove_all(texto, pattern = "@@\\S+")) |>
  # Elimina cualquier secuencia que empiece con "@@" seguida de caracteres sin espacio
  # (típicamente marcas o etiquetas de hablante incrustadas en el texto).

  step_mutate(texto = str_remove_all(texto, "\\d+")) |>
  # Elimina todos los dígitos (cifras, años, porcentajes).

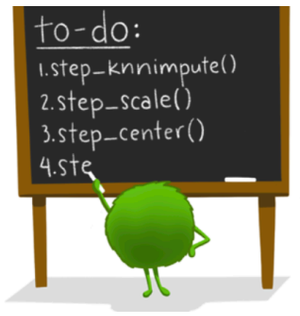
  step_tokenize(texto, token = "words") |>
  # Divide cada texto en tokens individuales (palabras). A partir de aquí la columna
  # `texto` deja de ser texto plano y pasa a ser una lista de tokens por documento.

  step_stopwords(texto, custom_stopword_source = stopwords_es) |>
  # Quita las palabras vacías (artículos, preposiciones, pronombres...) usando la lista
  # propia `stopwords_es` en lugar del diccionario por defecto del paquete.

  step_tokenfilter(texto, min_times = 3, max_tokens = 500) |>
  # Filtra el vocabulario: conserva solo tokens que aparecen al menos 3 veces en el
  # conjunto de entrenamiento y, de esos, se queda con los 500 más frecuentes.
  # Esto controla el tamaño de la matriz final y descarta palabras raras.

  step_tfidf(texto)
# Convierte los tokens en columnas numéricas: una columna por token del vocabulario
# (tfidf_texto_<palabra>), con el peso TF-IDF de cada palabra en cada documento.
```






# Ejercicio 01

## Ejercicio - Preparación del modelo

1. A partir de la matriz de documento-término del ejercicio anterior, obtenga la matriz de variables X y el vector de etiquetas Y necesarias para la base de entrenamiento del modelo de clasificación.
2. Utilizando *tidymodels*, programe el modelo de clasificación.
3. Evalúe, a partir de la matríz de confusión, la exactitud del modelo.

# Mejora de resultados del modelo

**Algunas cosas que se podría hacer para mejorar la exactitud del modelo:**

1. Modificar inteligentemente los stop
2. Generar un diccionario de palabras ofensivas
3. Ajustar la dispersión de la matríz dtm
4. Agregar **características adicionales** (más variables de X) a la matriz dtm. Como características de sentimientos.
5. Cambiar el modelo de clasificación o armar un ensamble de modelos.
6. Modificar los hiperparámetros.